

Auditoria de Performance

Projecto: Arquiv — Normas Técnicas de Construção
Data: 13 de Junho de 2026
Framework: Next.js 15.1.0 · React 19 · Supabase · Cloudflare Edge
Ficheiros analisados: ~40 ficheiros · ~12.000 linhas de código

1. Bundle & First Load JS

Rota	Page Size	First Load JS
/ (HomePage)	11.9 kB	185 kB
/upload	7.62 kB	168 kB
/login	6.54 kB	113 kB
/privacypolicy	6.99 kB	117 kB
/termosofuse	6.04 kB	116 kB
/about	2.96 kB	113 kB
/contact	1.5 kB	112 kB
/norm_detail/[normId]	2.13 kB	183 kB
/norm/[country]/[code]	2.1 kB	183 kB
Shared by all		
chunks/1517	50.7 kB	
chunks/4bd1b696	53 kB	
other shared chunks	2.76 kB	
Total shared	106 kB	
Middleware	88.2 kB (312 kB built)	

2. Problemas Críticos

● C1 — 14 chamadas AI por pesquisa

Ficheiro: `app/actions/norm-actions.ts` (linhas 1194–2178)

Pipeline actual por `searchNormsSemantic` (sem cache):

1. `interpretUserQuery(query)` → 1 call AI (google/gemma-4-31b-it:free)
2. `generateEmbedding(query)` → 1 call AI (openai/text-embedding-3-small) — *quebrado (HTTP 402)*
3. Summary relevance (Stage 1) → 1 call AI (claude-3.5-haiku)
4. `extractAnswerFromArticle` × 10 → 10 calls AI (claude-3.5-haiku)
5. `rankResultsWithAI` → 1 call AI (claude-3.5-haiku)

Total: 14 API calls externas por pesquisa. ~5–15s de latência.

Diagnóstico: A maior parte destas calls é redundante. O `interpretUserQuery` tem fallback `analyzeQueryIntent` (JS puro) quase equivalente. O embedding está quebrado (402); então o vector search cai sempre no pipeline AI lento. O `extractAnswerFromArticle` para 10 resultados é excessivo.

Recomendação:

- Usar `analyzeQueryIntent` (JS puro) como padrão; AI apenas para consultas ambíguas
- Reduzir `extractAnswerFromArticle` de 10 para 3 resultados
- Resolver o embedding 402 (pagar ou mudar de modelo) para usar vector search, que é mais rápido que o pipeline AI
- Se possível, substituir `crypto.subtle.digest('SHA-256')` por hash mais leve (FNV-1a ou similar)

● C2 — AIChatCard importado estaticamente no layout

Ficheiro: `app/layout.tsx:6`

AIChatCard (6.3 kB, 143 linhas, com framer-motion) é importado estaticamente no root layout. Isto significa que é **incluído no First Load JS de todas as páginas**, para todos os utilizadores, mesmo que nunca seja renderizado (só aparece para admins logados).

Impacto: +6.3 kB + framer-motion (~30 kB gzipped) em cada página.

Solução: Mudar para dynamic import com `ssr: false`

● C3 — Inline functions nas listas quebram React.memo

Ficheiros: `components/NormDisplay.tsx` `components/SemanticNormDisplay.tsx`

Ambos os componentes usam `React.memo` mas todos os handlers nas listas são inline arrow functions:

- `onClick={() => toggleExpanded(norm.id)}`
- `onClick={(e) => handleGenerateSummary(norm.id)}`
- `onClick={() => toggleContentExpanded(sectionId)}`

Impacto: Cada render recria dezenas de closures. O `React.memo` torna-se inútil porque os props (handlers) mudam sempre. Com listas de 10–20 normas, cada uma com 5–10 handlers, são 50–200 novas funções por render.

Solução: Envolver handlers em `useCallback` ou extrair para sub-componentes.

3. Problemas Altos

● A1 — Middleware 312 kB

Ficheiros: `middleware.ts` (46 linhas fonte) → `.next/server/middleware.js` (312 kB built)

Importa `@supabase/ssr` + `@supabase/supabase-js` completo (Auth, OAuth, PKCE, MFA, WebAuthn, JWT, Realtime) só para fazer `supabase.auth.getUser()`. Roda em **todas as rotas** excepto `api|_next/static|_next/image|favicon.ico|public`

Impacto: Páginas estáticas como `/about`, `/contact`, `/privacypolicy`, `/termosofuse` processam 312 kB de middleware desnecessariamente.

Solução:

- Adicionar `matcher` no middleware para incluir só as rotas que precisam de auth: `/upload` `/admin/*` `/norm_detail/*` `/api/*`
- OU substituir por JWT verification com biblioteca leve (`jose`)

● A2 — Cache em memória volátil

Ficheiros: `lib/cache-edge.ts`

Usa `Map` em `globalThis` → cache reinicia a cada cold start (deploy, worker restart). TTL de 5 min para search, 1h para static. Variação mínima na query (ex: "altura máxima" vs "altura maxima") gera cache miss.

Solução: Implementar cache persistente via Supabase KV ou Cloudflare KV. Normalizar queries (lowercase, remover acentos) antes de gerar cache key para aumentar hit rate.

● A3 — Stop words duplicadas 3x

Ficheiros:

- `lib/search-utils.ts:9-26` (canónico)
- `app/actions/norm-actions.ts:828-847` (duplicado em `extractBestSnippet`)

- `app/actions/norm-actions.ts:935-954` (duplicado em `extractAllSnippets`)

Risco: Alterar stop words num local não propaga para os outros, causando inconsistências de busca.

Solução: Extrair para `lib/stop-words.ts` e importar onde necessário.

4. Problemas Médios

M1 — Dead code (7 ficheiros/funções)

Ficheiro	Função/Ficheiro	Linhas	Motivo
<code>lib/cache.ts</code>	Ficheiro inteiro (10 exports)	102	Substituído por <code>lib/cache-edge.ts</code>
<code>lib/semantic-search.ts</code>	<code>analyzeDocumentStructure</code>	57–133	Substituído por <code>analyzeDocumentStructureServer</code>
<code>lib/semantic-search.ts</code>	<code>generateArticleInterpretation</code>	360–396	Nunca importado
<code>lib/gemini.ts</code>	<code>extractDocumentStructure</code>	323–392	Nunca importado
<code>lib/gemini.ts</code>	<code>deleteNorm</code>	470–478	Substituído por <code>deleteNormServer</code>
<code>lib/rate-limit.ts</code>	<code>resetRateLimit</code>	198	Nunca importado
<code>lib/google-search-console.ts</code>	<code>checkIndexingStatus</code>	45	Stub, nunca importado
<code>page.tsx</code> (raiz)	Ficheiro vazio	0	Não faz parte de rota Next.js

Impacto: ~400 linhas mortas que aumentam a superfície de manutenção e podem causar confusão.

M2 — 3 rotas debug expostas em produção

Rota	Ficheiro	Risco
<code>/api/debug-norms</code>	<code>app/api/debug-norms/route.ts</code> (146 linhas)	Expõe estrutura da DB, normas, categorias, países
<code>/api/test-search</code>	<code>app/api/test-search/route.ts</code> (58 linhas)	Expõe resultados brutos de busca com scores
<code>/api/test-supabase</code>	<code>app/api/test-supabase/route.ts</code> (161 linhas)	Expõe estrutura da DB via REST directo

Risco de segurança: Qualquer visitante pode aceder a estas rotas e obter informação interna da base de dados.

● **M3 — `processSearchQuery` chamado 4+ vezes por pesquisa**

Call sites em `norm-actions.ts` : linha 1358 (vector search), linha 1662 (AI pipeline), linha 2278 (fallback textual), além de dentro de `extractBestSnippet` (825) e `extractAllSnippets` (933). Cada chamada reprocessa o mesmo query — expansão de sinónimos, stemming, etc.

Solução: Chamar uma vez no início de `searchNormsSemantic` e passar como argumento para as funções internas.

● **M4 — Rota `/norm/[country]/[code]` duplicada com bug**

Ficheiro: `app/norm/[country]/[code]/page.tsx`

Renderiza o mesmo `NormDetailView` que `/norm_detail/[normId]` mas o sitemap só aponta para a versão ID. Tem `title=""` hardcoded (bug — o title da norma não é passado). Parece ser uma rota legada/deprecated.

Solução: Consolidar para uma rota canónica (manter `/norm_detail/[normId]`) e redireccionar a outra, ou fazer redirect 301.

● **M5 — `crypto.subtle.digest('SHA-256')` em `recordSearch`**

Ficheiro: `lib/rate-limit.ts:33-37`

Para cada pesquisa (mesmo cache hit), é gerado um hash SHA-256 async do IP + query + país. SHA-256 é desnecessariamente pesado para esta finalidade.

Solução: Substituir por FNV-1a (hash não-criptográfico) ou até uma string concatenada como cache key.

5. Problemas Baixos

● L1 — framer-motion em 10 componentes

Usado em: NormDetailView, NormDisplay, SemanticNormDisplay, action-search-bar, ai-chat, AuthModal, HomePage, search-loading, RateLimitModal.

Páginas estáticas (about, contact, privacy, terms) não precisam de framer-motion, mas ele está presente no layout via AIChatCard.

Solução: Dynamic import do AIChatCard (já cobre este problema). Para animações simples (fade-in de cards), CSS transitions são mais leves.

● L2 — IIFE no JSX do SemanticNormDisplay

Ficheiro: `components/SemanticNormDisplay.tsx:328`

`{{() => { ... }}()` — função executada imediatamente em cada render. Extrair para componente separado para melhor legibilidade e desempenho.

● L3 — 13 scripts de teste na raiz

`debug-*.js` `test-*.js` `find-*.js` `get-*.js` na raiz do projecto. Poluição visual. Devem estar em `scripts/`

● L4 — Dependência `node-cache` não utilizada

Instalada em `package.json` mas só usada pelo `lib/cache.ts` que está morto. Pode ser removida.

● L5 — Cobertura de código (testes)

Não existe framework de testes (jest, vitest). Os scripts de teste são manuais e inconsistentes. Considerar adicionar testes unitários para funções críticas (`processSearchQuery` , `generateVariants` , `searchNormisSemantic`).

6. Plano de Acção Recomendado

Prioridade	Acção	Ficheiros	Ganho estimado
1	Dynamic import AIChatCard no layout	<code>app/layout.tsx</code>	-6.3 kB + framer-motion no First Load JS de todas as páginas
2	Usar <code>analyzeQueryIntent</code> (JS) em vez de <code>interpretUserQuery</code> (AI)	<code>lib/search-utils.ts</code> , <code>app/actions/norm-actions.ts</code>	-1 AI call/pesquisa (~1.5s latência)
3	Reduzir <code>extractAnswerFromArticle</code> de 10 para 3	<code>app/actions/norm-actions.ts:2128</code>	-7 AI calls/pesquisa (~7s latência)
4	Adicionar <code>matcher</code> ao middleware	<code>middleware.ts</code>	312 kB não processado em páginas estáticas
5	<code>useCallback</code> nos handlers das listas	<code>components/NormDisplay.tsx</code> , <code>components/SemanticNormDisplay.tsx</code>	Menos re-renders em listas de 10–20 normas
6	Remover dead code (7 ficheiros/funções)	Vários (ver M1)	-400 linhas, menos confusão
7	Remover 3 rotas debug de produção	<code>app/api/debug-norms</code> , <code>test-search</code> , <code>test-supabase</code>	Segurança + -365 linhas
8	Centralizar stop-words	Criar <code>lib/stop-words.ts</code>	DRY, consistência de busca
9	Consolidar rotas de norma duplicadas	<code>app/norm/[country]/[code]</code>	Corrige bug do title,

			elimina rota morta
10	Substituir SHA-256 por hash leve no rate-limit	lib/rate-limit.ts:33	Menos CPU em cada pesquisa
11	Mover scripts de teste para scripts/	Raiz do projecto	Organização
12	Remover node-cache das dependências	package.json	Menos dependência não usada

7. Métricas de Build (Actuais)

Métrica	Valor
Total pages	11 estáticas + 10 dinâmicas
First Load JS shared	106 kB
Maior página (Home)	185 kB
Middleware	88.2 kB (312 kB built)
norm-actions.ts	2.145 linhas (god file)
Dependências totais	42 packages
Bibliotecas pesadas lazy-loadadas	pdfjs-dist (2.5 MB), react-quill-new (150 kB), mammoth (20 kB)

8. Conclusão

O Arquiv está estruturalmente sólido com boas práticas de lazy loading para bibliotecas pesadas. Os principais ganhos de performance vêm de:

1. **Reduzir chamadas AI redundantes** (eliminar 8 das 14 calls por pesquisa = ~57% menos latência)
2. **Otimizar o bundle do layout** (dynamic import AIChatCard = -6 kB + framer-motion)
3. **Limitar o middleware** a rotas que precisam de auth (312 kB não processado em páginas estáticas)
4. **Resolver o embedding 402** para voltar ao vector search (mais rápido que pipeline AI)

5. **Limpeza de dead code** (~400 linhas, 3 rotas debug, 1 dependência não usada)

Estas melhorias podem reduzir o tempo de pesquisa de 5–15s para 1–3s e cortar ~15% do First Load JS.